Operating System

Prepared By:

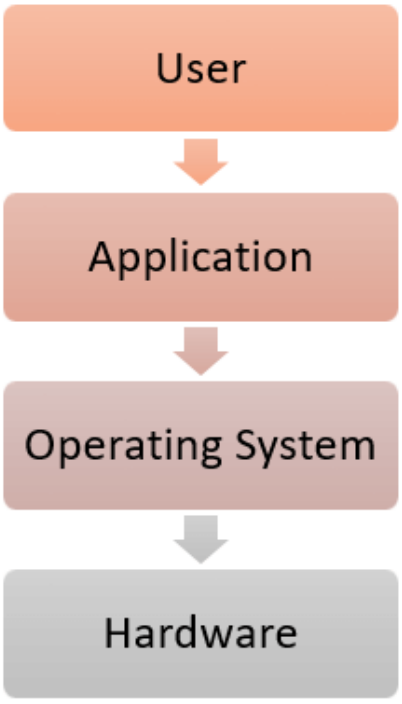
Er. Ayush Chaudhary

Chapter 1
Operating System Overview

Contents:

Definition:

- An operating system is a program that controls the execution of application programs and acts as an interface between the user of a computer and the computer hardware.
- An operating system acts as an intermediary between the user of a computer and computer hardware.
- The purpose of an operating system is to provide an environment in which a user can execute programs conveniently and efficiently.



Note: The first operating system was introduced in the early 1950's, it was called GMOS and was created by General Motors for IBM's machine the 701.

Examples of Operating Systems are –

- Windows, Linux, macOS(Macintosh Operating System), Android, iOS

Operating system has the following features:

1. **Convenience:** An OS makes a computer more convenient to use.
2. **Efficiency:** An OS allows the computer system resources to be used efficiently.
3. **Throughput:** An OS should be constructed so that It can give maximum **throughput**(Number of tasks per unit time).

Two View of the Operating System:

The operating system can be observed from the point of view of the user or the system.

This is known as the user view and system view respectively.

An operating system is a framework that enables user application programs to interact with system hardware.

There are two primary views of an operating system:

1. User view
2. System view

1. **User View:**
    - **Graphical User Interface (GUI):**

      Many modern operating systems, such as Windows, macOS, and various Linux distributions, provide a graphical user interface.

      Users interact with the system through visual elements like icons, windows, menus, and buttons.

- **Command-Line Interface (CLI):**

  Some users prefer to interact with the operating system through a text-based command-line interface.

  This involves typing commands to perform tasks. Examples include the Command Prompt in Windows, Terminal in macOS, and various shells in Linux.

- **Applications:**

  Users primarily engage with the operating system through applications.

  These applications can be productivity tools, games, web browsers, or any software that runs on the operating system.

2. **System View:**

- **Resource Management:**

  From a system perspective, the operating system manages hardware resources such as CPU, memory, disk space, and peripherals.

It ensures that different programs and processes get the necessary resources without interfering with each other.

- **Process and Memory Management:**

  The operating system is responsible for managing processes, which are executing programs, and allocating memory to them.

  It ensures efficient use of resources and prevents one process from accessing another's memory space.

- **File System Management:**

  The operating system provides a file system that organizes and stores data on storage devices.

  It manages files, directories, and file permissions, ensuring data integrity and security.

- **Device Drivers:**

  The operating system includes device drivers that enable communication between the computer's hardware components and the software.

  These drivers facilitate the interaction between the operating system and devices like printers, scanners, and graphics cards.

- **Security and Authentication:**

  The operating system implements security features to protect data and resources.

  This includes user authentication, access control, encryption, and antivirus measures.

Generation/History of operating system:

First Generation (1940 – 1955):

- When the first electronic computer was developed in 1940, it was created without any operating system.
- In early times, users have full access to the computer machine and write a program for each task in absolute machine language.
- The programmer can perform and solve only simple mathematical calculations during the computer generation, and this calculation does not require an operating system.

Second Generation (1955 – 1965):

- The second-generation operating system was based on a single stream batch processing system because it collects all similar jobs in groups or batches and then submits the jobs to the operating system using a punch card to complete all jobs in a machine.

Third Generation (1965 – 1980):

- During the late 1960s, operating system designers were very capable of developing a new operating

system that could simultaneously perform multiple tasks in a single computer program called multiprogramming.

Fourth Generation (1980 - Present Day):

- The fourth generation of operating systems is related to the development of the personal computer.

Generation	Year	Electronic device used	Types of OS Devices
First	1945-55	Vacuum Tubes	Plug Boards
Second	1955-65	Transistors	Batch Systems
Third	1965-80	Integrated Circuits(IC)	Multiprogramming
Fourth	Since 1980	Large Scale Integration	PC

Function of operating system:

1. Resource Management:

- When parallel accessing happens in the OS means when multiple users are accessing the system the OS works as Resource Manager.
- Its responsibility is to provide hardware to the user. It decreases the load in the system.

2. Process Management:

- A process is a program in execution.
- A process needs certain resources, including CPU time, memory, files, and I/O devices, to accomplish its task.
- The operating system is responsible for the following activities in connection with process management.
  -Process creation and deletion.
  -process suspension and resumption.
  -Provision of mechanisms for:
    * process synchronization
    * process communication

3. Main-Memory Management:

- Main memory is a volatile storage device. It loses its contents in the case of system failure.
- The operating system is responsible for the following activities in connection with memory management:
  - –Keep track of which parts of memory are currently being used and by whom.
  - –Decide which processes to load when memory space becomes available.
  - –Allocate and deallocate memory space as needed.

4. Secondary-Storage Management:

- Since main memory (primary storage) is volatile and too small to accommodate all data and programs permanently, the computer system must provide secondary storage to back up main memory.
- Most modern computer systems use disks as the principle on-line storage medium, for both programs and data.
- The operating system is responsible for the following activities in connection with disk management:

–Free space management

–Storage allocation

–Disk scheduling.

5. Security/Privacy Management:

- Privacy is also provided by the Operating system by means of passwords so that unauthorized applications can't access programs or data.
- For example, Windows uses **Kerberos** (Kerberos is a network authentication protocol that provides secure communication over an insecure network) authentication to prevent unauthorized access to data.

6. File Management:

- A file is a collection of related information defined by its creator.
- Commonly, files represent programs (both source and object forms) and data.
- The operating system is responsible for the following activities in connection with file management:

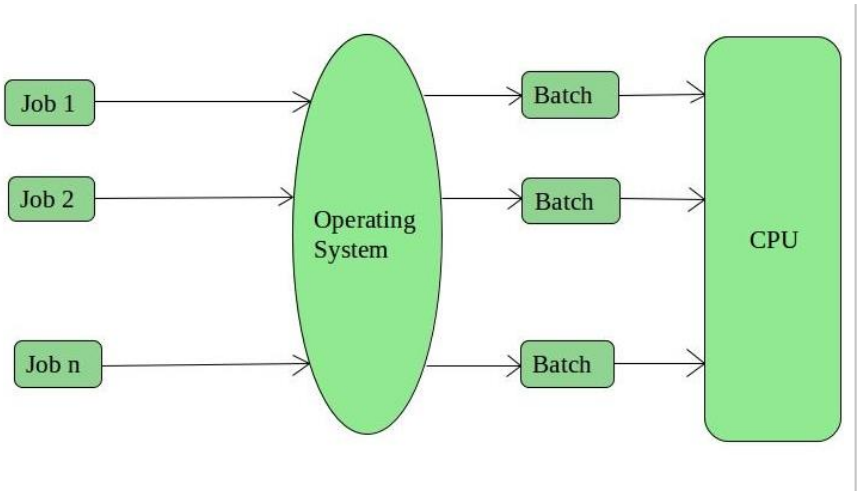
  –File creation and deletion.

–Directory creation and deletion.
–Mapping files onto secondary storage.
–File backup on stable (nonvolatile) storage media.

Types of Operating System:

1. Batch Operating System:

- This type of operating system does not interact with the computer directly.
- There is an operator which takes similar jobs having the same requirement and groups them into batches.
- It is the responsibility of the operator to sort jobs with similar needs.



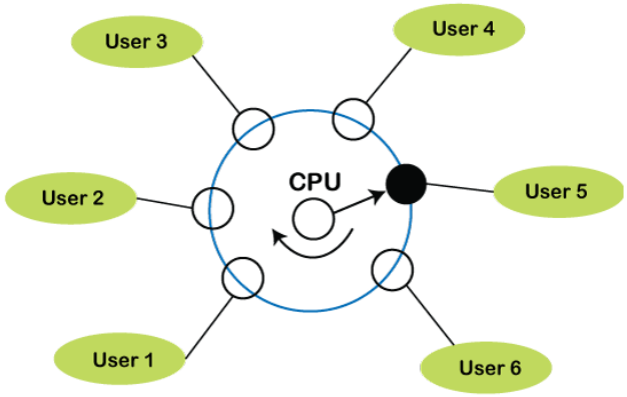
- The batch operating system **runs all the jobs as a non-preemptive process.**

2. Multi programming:

- Multi-programming increases CPU utilization by organizing jobs (code and data) so that the CPU always has one to execute.
- The idea is to keep multiple jobs in main memory.
- If one job gets occupied with IO, CPU can be assigned to other job.

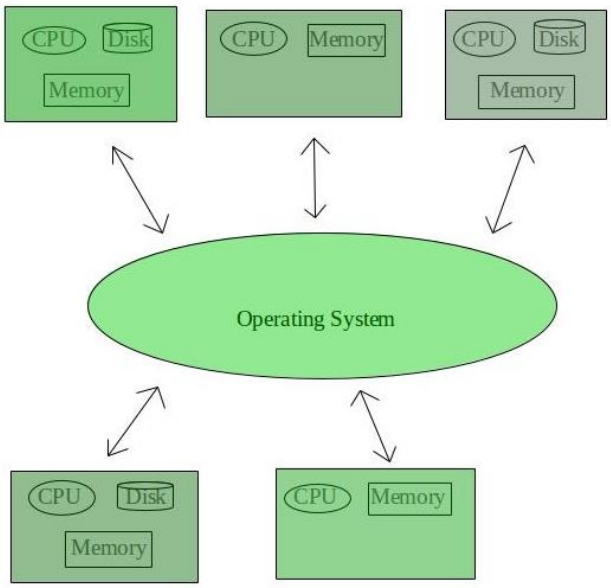
3. Time-Sharing Operating Systems:

- Each task is given some time to execute so that all the tasks work smoothly.
- These systems are also known as Multitasking Systems.
- The time that each task gets to execute is called quantum.
- After this time interval is over OS switches over to the next task.



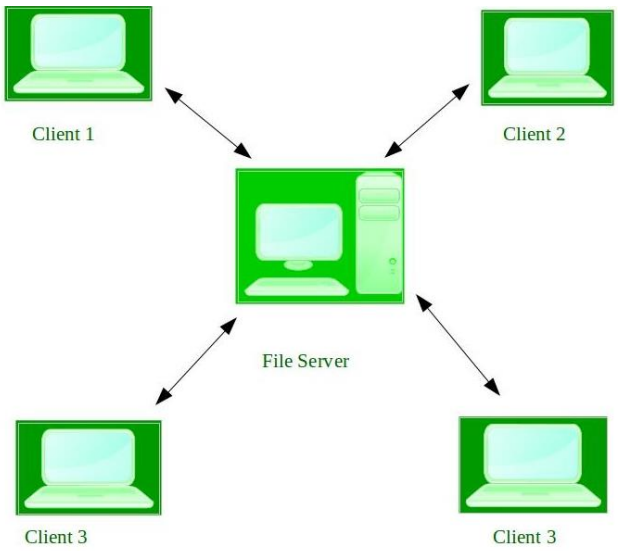
4. Distributed Operating System:

- A distributed operating system provides an environment in which multiple independent CPU or processor communicates with each other through physically separate computational nodes.
- These types of the operating system is a recent advancement in the world of computer technology and are being widely accepted all over the world and, that too, with a great pace.



5. Network Operating System:

- These systems run on a server and provide the capability to manage data, users, groups, security, applications, and other networking functions.
- These types of operating systems allow shared access of files, printers, security, applications, and other networking functions over a small private network.



6. Real-Time Operating System:

- These types of OSs serve real-time systems.
- The time interval required to process and respond to inputs is very small.
- This time interval is called **response time**.
- **Real-time systems** are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots, etc.

Two types of Real-Time Operating System which are as follows:

- Hard Real-Time Systems
- Soft Real-Time Systems

Operating System as Resource Manager:

Let us understand how the operating system works as a Resource Manager.

- Now-a-days all modern computers consist of processors, memories, timers, network interfaces, printers, and so many other devices.
- The operating system provides for an orderly and controlled allocation of the processors, memories, and I/O devices among the various programs in the bottom-up view.
- Operating system allows multiple programs to be in memory and run at the same time.
- Resource management includes multiplexing or sharing resources in two different ways: in time and in space.
- In time multiplexed, different programs take a chance of using CPU. First one tries to use the resource, then the next one that is ready in the queue and so on. For example: Sharing the printer one after another.

Operating System as Extended Machine:

Let us understand how the operating system works as an Extended Machine.

- At the Machine level the structure of a computer's system is complicated to program, mainly for input or output. Programmers do not deal with hardware. They will always mainly focus on implementing software. Therefore, a level of abstraction is supposed to be maintained.
- Operating systems provide a layer of abstraction for using disk such as files.
- This level of abstraction allows a program to create, write, and read files, without dealing with the details of how the hardware actually works.
- The level of abstraction is the key to managing the complexity.
- The first is to define and implement the abstractions.
- The second is to solve the problem at hand.

System Calls:

A system call is a programmatic way in which a computer program requests a service from the kernel of the operating system it is executed on.

A system call is a way for programs to interact with the operating system.

A computer program makes a system call when it makes a request to the operating system's kernel.

System call provides the services of the operating system to the user programs via Application Program Interface(API).

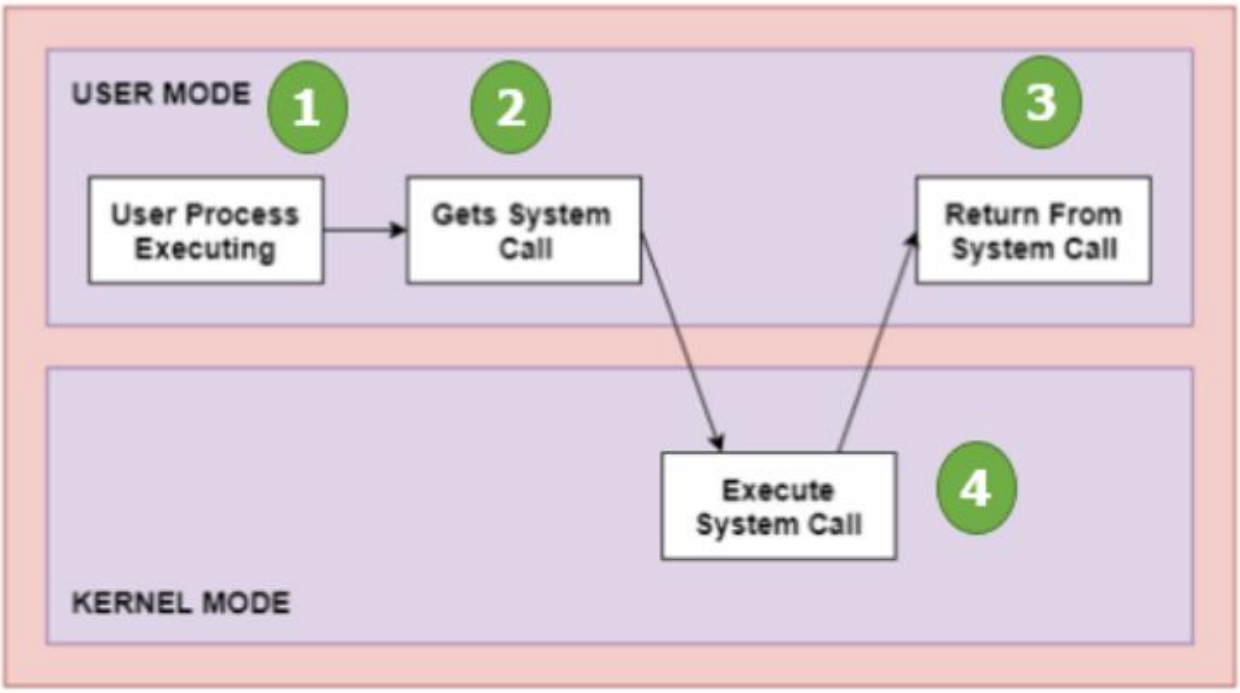
How are system calls made?

When a computer software needs to access the operating system's kernel, it makes a system call.

The system call uses an API to expose the operating system's services to user programs.

It is the only method to access the kernel system.

All programs or processes that require resources for execution must use system calls, as they serve as an interface between the operating system and user programs.



Why do you need system calls in Operating System?

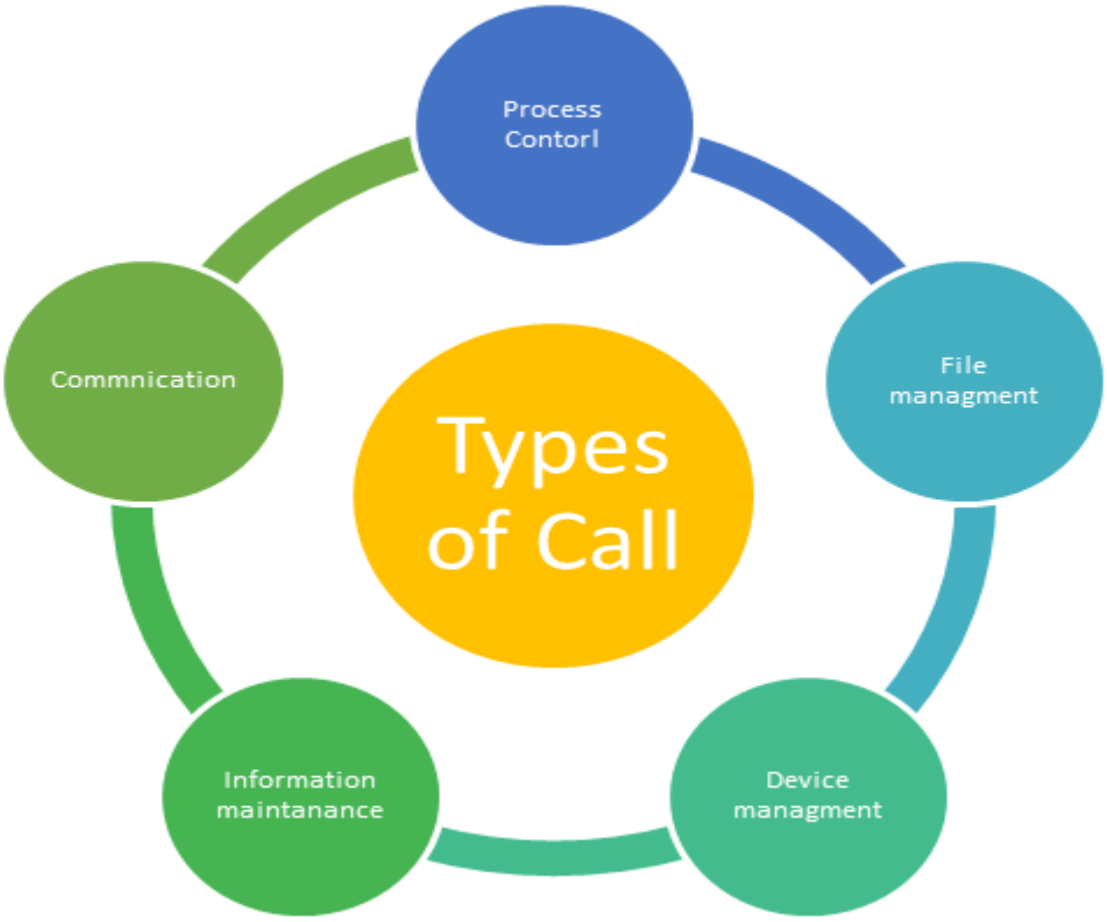
There are various situations where you must require system calls in the operating system. Following of the situations are as follows:

1. It is must require when a file system wants to create or delete a file.
2. If you want to read or write a file, you need to system calls.
3. If you want to access hardware devices, including a printer, scanner, you need a system call.
4. System calls are used to create and manage new processes.

Types of System calls

Here are the five types of System Calls in OS:

- Process Control
- File Management
- Device Management
- Information Maintenance
- Communications



Process Control

This system calls perform the task of process creation, process termination, etc.

Functions:

- End and Abort
- Load and Execute
- Create Process and Terminate Process
- Wait and Signal Event
- Allocate and free memory

File Management

File management system calls handle file manipulation jobs like creating a file, reading, and writing, etc.

Functions:

- Create a file
- Delete file
- Open and close file
- Read, write, and reposition
- Get and set file attributes

Device Management

Device management does the job of device manipulation like reading from device buffers, writing into device buffers, etc.

Functions:

- Request and release device
- Logically attach/ detach devices
- Get and Set device attributes

Information Maintenance

It handles information and its transfer between the OS and the user program.

Functions:

- Get or set time and date
- Get process and device attributes

Communication

These types of system calls are specially used for interprocess communications.

Functions:

- Create, delete communications connections
- Send, receive message
- Help OS to transfer status information
- Attach or detach remote devices

Categories	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
Device manipulation	SetConsoleMode() ReadConsole() WriteConsole()	loctl() read() write()
File manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	Open() Read() write() close!)
Information maintanence	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	Pipe() shm_open() mmap()

Handling System Call:

1. **User-Space to Kernel-Space Transition:**
    - When a user-level process needs to perform a privileged operation or access a system resource, it makes a system call. This is typically done through a programming language-specific mechanism, such as a software interrupt or a specific instruction (e.g., **int 0x80** in x86 assembly).
    - The transition from user space to kernel space is a critical part of handling system calls. It involves switching from the user mode, where applications execute, to the kernel mode, where the operating system has higher privileges.
2. **System Call Dispatcher:**
    - Once the system call is initiated, the operating system's kernel takes control. The system call dispatcher, a component of the kernel, identifies the type of system call being requested based on a provided identifier or code.
3. **Parameter Passing:**
    - System calls often require parameters to specify details of the requested operation. These parameters are passed from the user space to the

kernel space, and the system call dispatcher extracts them.

4. **Execution of System Call Handler:**
    - The system call dispatcher transfers control to the appropriate system call handler. Each system call has a corresponding handler or routine within the kernel that knows how to perform the requested operation.
5. **Execution of Kernel Code:**
    - The kernel code executes within the kernel space, where it has direct access to system resources and hardware. The system call handler performs the requested operation on behalf of the user-level process.
6. **Result Passing:**
    - After the system call handler completes its execution, the result is returned to the user space. This could be a return value, an error code, or any relevant information indicating the outcome of the system call.

7. **Return to User Space:**
    - The operating system switches the execution mode back to user space, and control returns to the original user-level process. The process continues its execution with the results obtained from the system call.

System Program:

The concept of system programs is an integral part of the logical hierarchy of a computer system, positioned between the operating system (OS) and application programs.

System programs play a crucial role in providing a convenient and efficient environment for program development and execution.

1. **Position in the Logical Hierarchy:**
    - System programs are positioned between the operating system and application programs in the logical hierarchy of a computer system.

- While the operating system manages the overall hardware and resources, system programs serve as intermediaries that facilitate the interaction between the operating system and application programs.

2. **Examples of System Programs:**
   - System utilities and command interpreters are examples of system programs.
   - These include a variety of tools and applications that assist users and programmers in managing the system, executing commands, and developing software.

3. **Providing a Convenient Environment:**
   - System programs contribute to creating a convenient and user-friendly environment for both program development and execution.
   - They offer a set of tools and utilities that simplify tasks related to file management, status information retrieval, programming language support, program loading, and communication.

4. **Categories of System Programs:**
    - **File Management:**
        - System programs in this category handle operations related to files and directories. Examples include copying, moving, deleting files, and creating or renaming directories.
    - **Status Information:**
        - These programs provide information about the system status, including the date, time, available disk space, and details about users currently logged into the system.
    - **Programming Language Support:**
        - System programs in this category support the development and execution of programs written in various programming languages. This includes interpreters for scripting languages, compilers for high-level languages, and assemblers for low-level languages.
    - **Program Loading and Support:**
        - Programs in this category, such as loaders and linkers, handle the process of loading

programs into memory for execution and linking different parts of a program together.

- **Communication:**
  - System programs supporting communication include tools for remote login (e.g., SSH), email handlers, and file transfer utilities. These programs enable users to connect to remote systems, manage email, and transfer files between systems.

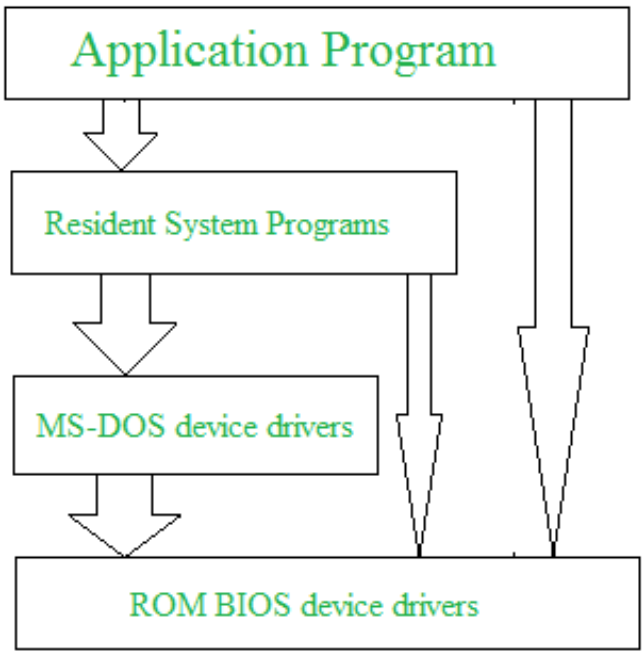
Operating System Structures:

Introduction:

- Operating system can be implemented with the help of various structures.
- The structure of the OS depends mainly on how the various common components of the operating system are interconnected into the kernel.
- Depending on this we have the following structures of the operating system:

1. Simple Structure:

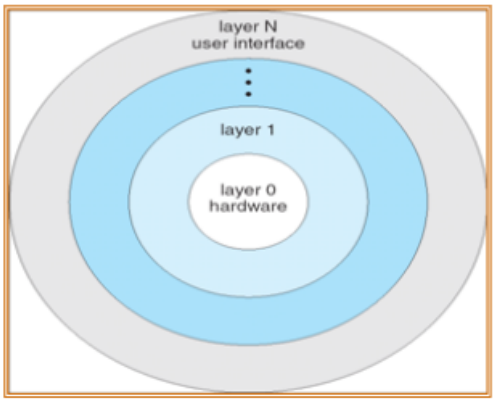
- Such operating systems do not have well defined structure and are small, simple and limited systems.
- The interfaces and levels of functionality are not well separated.
- MS-DOS is an example of such operating system.
- In MS-DOS application programs are able to access the basic I/O routines.
- These types of operating system cause the entire system to crash if one of the user programs fails.



2. Layered structure:

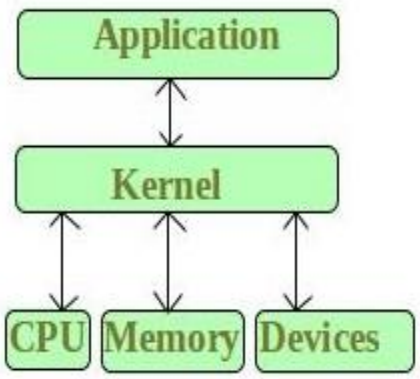
- An operating system with different layers for handling system software and user software is known as a layered operating system.
- In this structure the OS is broken into a number of layers (levels).
- The bottom layer (layer 0) is the hardware and the topmost layer (layer N) is the user interface.
- These layers are so designed that each layer uses the functions of the lower-level layers only.
- This simplifies the debugging process as if lower-level layers are debugged and an error occurs during debugging then the error must be on that layer only as the lower level layers have already been debugged.

layer 5:	user programs
layer 4:	buffering for input and output
layer 3:	Process management
layer 2:	memory management
layer 1:	CPU scheduling
layer 0:	hardware



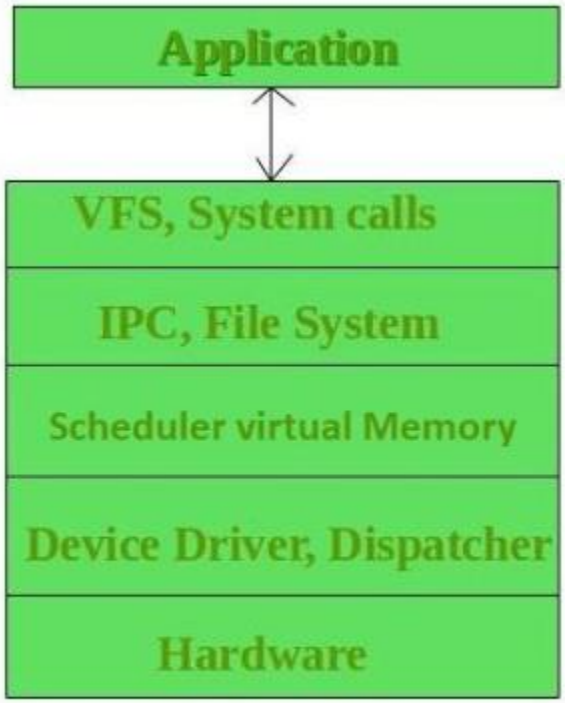
Kernel:

- Kernel is central component of an operating system that manages operations of computer and hardware.
- It basically manages operations of memory and CPU time. It is core component of an operating system.
- Kernel acts as a bridge between applications and data processing performed at hardware level using inter-process communication and system calls.
- Kernel loads first into memory when an operating system is loaded and remains into memory until operating system is shut down again.
- It is responsible for various tasks such as disk management, task management, and memory management.



Monolithic Kernels:

- In a monolithic kernel, the same memory space is used to implement user services and kernel services.



Monolithic Kernel

- It means, in this type of kernel, there is no different memory used for user services and kernel services.
- As it uses the same memory space, the size of the kernel increases, increasing the overall size of the OS.
- The execution of processes is also faster than other kernel types as it does not use separate user and kernel space.

Examples of Monolithic Kernels are Unix, Linux,etc.

Advantages:

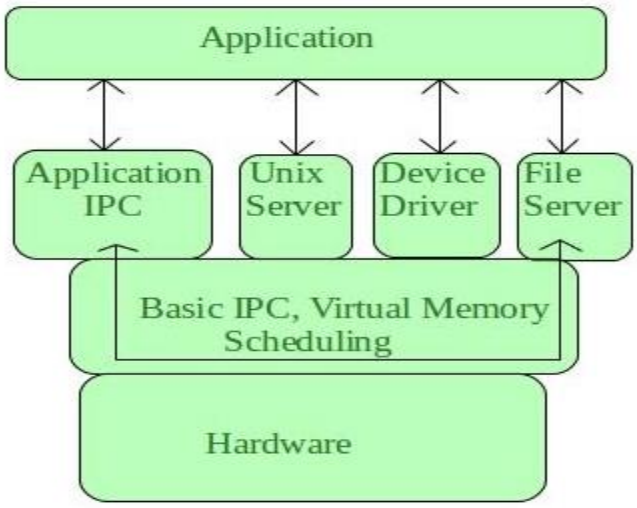
- The execution of processes is also faster as there is no separate user space and kernel space.
- As it is a single piece of software hence, it's both sources and compiled forms are smaller.

Disadvantages:

- If any service generates any error, it may crash down the whole system.
- Large in size and hence become difficult to manage.
- To add a new service, the complete operating system needs to be modified.

Micro Kernel:

- It is different from a traditional kernel or Monolithic Kernel.
- In this, **user services and kernel services are implemented into two different address spaces: user space and kernel space**.



- Since it uses different spaces for both the services, so, the size of the microkernel is decreased, and which also reduces the size of the OS.
- Microkernels are easier to manage and maintain as compared to monolithic kernels.
- Still, if there will be a greater number of system calls and context switching, then it might reduce the performance of the system by making it slow.
- These kernels use a message passing system for handling the request from one server to another server.

Examples of Microkernel are Mac OS X etc.

Advantages

- Microkernels can be managed easily.
- A new service can be easily added without modifying the whole OS.

- In a microkernel, if a kernel process crashes, it is still possible to prevent the whole system from crashing.

Disadvantages

- Process management is very complicated.
- The messaging bugs are difficult to fix.

ExoKernel:

- Exokernel is still developing and is the experimental approach for designing OS.
- This type of kernel is different from other kernels as in this; resource protection is kept separated from management, which allows us to perform application-specific customization.

Advantages:

- The exokernel-based system can incorporate multiple library operating systems. Each library exports a different API, such as one can be used for high-level UI development, and the other can be used for real-time control**.**

Disadvantages:

- The design of the exokernel is very complex.

S.N.	Micro Kernel	Monolithic Kernel
1.	In microkernel, user services and kernel services are kept in separate address space.	In monolithic kernel, both user services and kernel services are kept in the same address space.
2.	Microkernel are smaller in size.	Monolithic kernel is larger than microkernel.
3.	Easier to add new functionalities.	Difficult to add new functionalities.
4.	OS is complex to design.	OS is easy to design and implement.
5.	To design a microkernel, more code is required.	Less code when compared to microkernel
6.	Execution speed is low.	Execution speed is high.
7.	Example: Mac OS X.	Example: Microsoft Windows 95.

Shell:

Shell is a program that serves as the interface between the user and the operating system.

It allows users to interact with the system by entering commands, which are then interpreted and executed by the operating system.

Shell accepts human-readable commands from the user and converts them into something which kernel can understand.

Types of Shell:

- **The C Shell –**
  - Denoted as **csh**
  - Bill Joy created it at the University of California at Berkeley.
  - It includes helpful programming features like built-in arithmetic and C-like expression syntax.

  In C shell:

  Command full-path name is /bin/csh,

  Non-root user default prompt is hostname %,

  Root user default prompt is hostname #.

- **The Bourne Shell –**
  - Denoted as **sh**
  - It was written by Steve Bourne at AT&T Bell Labs.
  - It is the original UNIX shell.
  - It is faster and more preferred.

  For the Bourne shell the:

  Command full-path name is /bin/sh and /sbin/sh,

  Non-root user default prompt is $,

  Root user default prompt is #.

- **The Korn Shell**
  - It is denoted as **ksh**
  - It Was written by David Korn at AT&T Bell Labs. It is a superset of the Bourne shell.So it supports everything in the Bourne shell.
  - It has interactive features. It includes features like built-in arithmetic and C-like arrays, functions, and string-manipulation facilities.
  - It is faster than C shell.

  For the Korn shell the:

  Command full-path name is /bin/ksh,

Non-root user default prompt is $,

Root user default prompt is #.

Open-source operating systems:

Open-source operating systems are software systems whose source code is made freely available to the public, allowing anyone to view, modify, and distribute the code.

This stands in contrast to closed-source or proprietary operating systems, where the source code is not accessible, and users can only obtain and use the compiled binary version.

1. **Source Code Availability:**
   - Open-source operating systems provide access to their source code, allowing developers to study how the system works, make modifications, and contribute improvements.
   - This transparency fosters collaboration and community-driven development.
2. **Counter to Copy Protection and DRM:**
   - Open-source operating systems are often seen as a response to the copy protection and Digital Rights Management (DRM) movements that restrict user access and control over software.

- The open-source philosophy advocates for user freedom, collaboration, and sharing of knowledge.

3. **Free Software Foundation (FSF) and GNU Public License (GPL):**
   - The Free Software Foundation (FSF) has been a key player in promoting open-source software.
   - The GNU Project, initiated by the FSF, developed a set of essential software tools and utilities, including the GNU General Public License (GPL).
   - The GPL is a "copyleft" license that ensures software derived from GPL-licensed code must also be open-source.
4. **Examples of Open-Source Operating Systems:**
   - **GNU/Linux:** A popular open-source operating system kernel (Linux) combined with GNU utilities. Various distributions (like Ubuntu, Fedora, and Debian) package and distribute GNU/Linux.
   - **BSD UNIX:** Berkeley Software Distribution (BSD) UNIX is another family of open-source operating systems. FreeBSD, NetBSD, and

OpenBSD are examples. The core of Mac OS X (now macOS) is based on a modified version of FreeBSD.

- **Sun Solaris:** While not as widely used as some other open-source systems, Sun Solaris has been open-sourced as OpenSolaris. The Illumos project, derived from OpenSolaris, continues development.

5. **Community Collaboration:**
    - Open-source operating systems benefit from a global community of developers who contribute to their improvement. Collaboration occurs through online forums, mailing lists, and version control systems.
    - This community-driven approach often leads to rapid bug fixes, security enhancements, and feature development.
6. **Customization and Flexibility:**
    - Users and organizations can customize open-source operating systems to suit their specific needs.

- This flexibility is a significant advantage, especially for enterprises and individuals with unique requirements.